Avaliação de Performance e Consumo Energético de C# em Windows e Linux: Um Estudo Comparativo com o Algoritmo FannkuchRedux

Gustavo Castro
Universidade do Minho, Departamento de Informática
Mestrado em Engenharia Informática

Maio de 2026

Conteúdo

1 Resumo **3**

2 Introdução **3**

3 Metodologia **3**
3.1 Algoritmo de Benchmark — FannkuchRedux . . . 3
3.2 Hardware e Software . . . 4
3.3 Medição de Performance — BenchmarkDotNet . . . 4
3.4 Medição de Energia — Intel RAPL . . . 4
3.5 Configuração do Ambiente de Medição . . . 5
3.6 Protocolo de Recolha de Dados . . . 5
3.7 Análise Estatística . . . 5

4 Resultados **6**
4.1 Estatísticas Descritivas . . . 6
4.2 Visualização dos Dados . . . 6
4.3 Comparação de Performance (Tempo) . . . 8
4.4 Comparação de Consumo Energético . . . 8
4.5 Significância Estatística . . . 8
4.6 Correlação Tempo–Energia . . . 9
4.7 Variância e Estabilidade . . . 9

5 Discussão **10**
5.1 Windows é mais rápido — porquê? . . . 10
5.2 Linux consome menos energia — porquê? . . . 10
5.3 Trade-off velocidade vs. energia . . . 11

6 Ameaças à Validade **11**
6.1 Validade Interna 11
6.2 Validade Externa 11
6.3 Validade de Construção 11

7 Conclusão **11**

1 Resumo

Este trabalho compara a performance e o consumo energético de C# em Windows 11 e Ubuntu 24.04 utilizando o algoritmo FannkuchRedux como *benchmark* CPU-bound. A medição de energia foi realizada diretamente via Intel RAPL (*Running Average Power Limit*), a medição de tempo via BenchmarkDotNet, e ambos os sistemas operativos correram no mesmo hardware (dual-boot), com turbo desativado para garantir comparabilidade. Os resultados mostram que Windows é 38.1% mais rápido para N=11 e 35.2% mais rápido para N=12, mas que Linux consome 69% menos energia para N=11 (15.99 J vs. 27.06 J) e 32% menos para N=12 (216.14 J vs. 285.03 J). Todas as diferenças são estatisticamente significativas (Mann-Whitney U, $p < 0.001$), confirmadas por testes de permutação independentes. Conclui-se que existe um trade-off claro entre velocidade e eficiência energética nos dois sistemas operativos, com implicações práticas para decisões de *deployment* em ambientes onde o custo energético é relevante.

2 Introdução

A eficiência energética do software é uma preocupação crescente, especialmente em contextos de computação de alto desempenho e *edge computing*. A linguagem C# e o ecossistema .NET são amplamente usados em ambientes de produção que correm tanto em Windows como em Linux. Compreender como o sistema operativo influencia a performance e o consumo energético de um mesmo programa é relevante para decisões de *deployment*.

A maioria dos estudos de performance em .NET foca-se em métricas de tempo de execução, ignorando o consumo energético. Estudos como o de Pereira et al. [5] (*greensoftwarelab/Energy Languages* [2]) demonstram que a linguagem e o ambiente de execução têm impacto significativo na energia consumida. Este trabalho estende essa linha de investigação para comparar dois sistemas operativos com o mesmo hardware e runtime.

Este trabalho tem como objetivos: (1) medir e comparar o tempo de execução de C# em Windows 11 e Ubuntu 24.04; (2) medir e comparar o consumo energético via Intel RAPL em ambos os sistemas operativos; (3) avaliar a significância estatística das diferenças observadas; (4) identificar possíveis causas para as diferenças encontradas.

O restante documento está organizado da seguinte forma: a Secção 3 descreve a metodologia e o ambiente experimental; a Secção 4 apresenta os resultados obtidos; a Secção 5 discute as implicações dos resultados; a Secção 6 identifica ameaças à validade; a Secção 7 conclui o trabalho.

3 Metodologia

3.1 Algoritmo de Benchmark — FannkuchRedux

O FannkuchRedux é um algoritmo CPU-bound puro, sem I/O, baseado em permutações de arrays, proveniente do *Computer Language Benchmarks Game* (CLBG) [6]. A implementação em C# utilizada é a referência de *greensoftwarelab/Energy-Languages* [2], contribuída por Isaac Gouy, transliterada do programa Java de Oleg Mazurov, com melhorias de concorrência por Peperud [5].

Foram testadas duas variantes: N=11 e N=12 (tamanho do problema, que determina a complexidade fatorial). O algoritmo utiliza *threading* via `Task.Run` com `Environment.ProcessorCount`

`+ 1` *workers*. A correção foi verificada: `Compute(7)` devolve *checksum* 228 e *maxFlips* 16, correspondendo ao valor de referência do CLBG. A alocação de *heap* é de aproximadamente 52 KB por iteração (verificado via `MemoryDiagnoser` do BenchmarkDotNet), pelo que o GC é irrelevante para os resultados.

3.2 Hardware e Software

A Tabela 1 resume as especificações do ambiente de teste.

Tabela 1: Especificações de hardware e software do ambiente de teste.

Campo	**Valor**
Máquina	HP EliteBook 1050 G1
CPU	Intel Core i7-8850H (Coffee Lake, 6C/12T, 2.6 GHz base)
RAM	16 GiB
Armazenamento	Samsung NVMe 512 GB
SO Linux	Ubuntu 24.04.3 LTS, kernel 6.17.0-22-generic
SO Windows	Windows 11 Pro (build 26200.8246)
.NET SDK	9.0.312 (Linux) / 9.0.313 (Windows)
Runtime .NET	9.0.14 (Linux) / 9.0.15 (Windows)
BenchmarkDotNet	0.15.8

3.3 Medição de Performance — BenchmarkDotNet

A ferramenta BenchmarkDotNet 0.15.8 [1] foi usada para medir o tempo de execução. A configuração incluiu `SimpleJob(RuntimeMoniker.Net90)`, `MemoryDiagnoser` e `CsvMeasurementsExpor` O aquecimento (*warm-up*) é realizado automaticamente antes das iterações medidas, com deteção e remoção automática de *outliers* estatísticos pelo BenchmarkDotNet. A compilação foi feita em modo *Release*, x64, com `Optimize=true`. Em Windows foi usado `InProcessEmitToolchain` (necessário para a integração com a LibreHardwareMonitorLib).

3.4 Medição de Energia — Intel RAPL

A energia foi medida via Intel RAPL [3] no domínio *Package* (`MSR_PKG_ENERGY_STATUS`, 0x611), que inclui todos os núcleos, e no domínio secundário PP0/*Cores* (`MSR_PP0_ENERGY_STATUS`, 0x639).

Linux: leitura via `/sys/devices/virtual/powercap/intel-rapl/intel-rapl:0/energy_uj` (sysfs, kernel powercap).

Windows: leitura via driver WinRing0 (LibreHardwareMonitorLib 0.9.3) [4], com acesso direto ao MSR por reflexão sobre `LibreHardwareMonitor.Hardware.Ring0.ReadMsr`.

A unidade de energia é lida do `MSR_RAPL_POWER_UNIT` (0x606), bits [12:8] — para Coffee Lake: 2^{-14} J/unit $\approx$ 61 µJ/unit. O método de medição consiste na leitura do contador antes e depois de cada iteração; o delta é a energia consumida. O *wraparound* do contador de 32 bits foi tratado (período de *wrap* muito superior à duração das *runs*).

3.5 Configuração do Ambiente de Medição

Para garantir comparabilidade entre os dois sistemas operativos, as seguintes configurações foram aplicadas:

Ambos os SO:

- Turbo Boost desativado (Linux: `/sys/devices/system/cpu/intel_pstate/no_turbo=1`; Windows: `PROCTHROTTLEMAX=99%`)
- Máquina ligada à corrente (AC power)
- Sem outras aplicações intensivas em CPU durante as medições

Linux:

- CPU governor: `performance` (via cpupower)
- Script de lançamento: `scripts/run-linux.sh`

Windows:

- Plano de energia: Desempenho Máximo (GUID `202e5b95...`)
- Processo executado como Administrador (necessário para acesso MSR via WinRing0)

3.6 Protocolo de Recolha de Dados

O número de iterações foi determinado automaticamente pelo BenchmarkDotNet (mínimo de estabilidade estatística). As amostras válidas, após remoção de *outliers* (critério IQR 1.5×), foram:

- Linux N=11: 23 amostras; Linux N=12: 20 amostras
- Windows N=11: 25 amostras; Windows N=12: 22 amostras

Os dados foram exportados para CSV com *timestamp*, energia (µJ), duração (ms) e temperatura.

3.7 Análise Estatística

Foram utilizados os seguintes testes e medidas:

- **Mann-Whitney U** (amostras independentes, não-paramétrico)
- **A/B test por permutação** (10 000 permutações)
- **Correlação de Spearman** ρ (relação não-linear entre tempo e energia)
- **Intervalos de confiança** a 95%, calculados via distribuição t de Student

O nível de significância foi $\alpha = 0.05$. As ferramentas utilizadas foram Python 3, pandas, scipy, seaborn e matplotlib.

4 Resultados

4.1 Estatísticas Descritivas

A Tabela 2 apresenta as estatísticas descritivas completas para os quatro grupos experimentais.

Tabela 2: Estatísticas descritivas por SO e valor de N.

SO	N	n	Média (ms)	Mediana (ms)	DP (ms)	CV (%)	Energia média (J)
Linux	11	23	623.5	623.7	3.95	0.63	15.99
Linux	12	20	8577.8	8578.3	28.10	0.33	216.14
Windows	11	25	451.5	451.7	4.28	0.95	27.06
Windows	12	22	6343.6	6335.2	34.64	0.55	285.03

4.2 Visualização dos Dados

A Figura 1 mostra a distribuição do tempo de execução por SO e N.

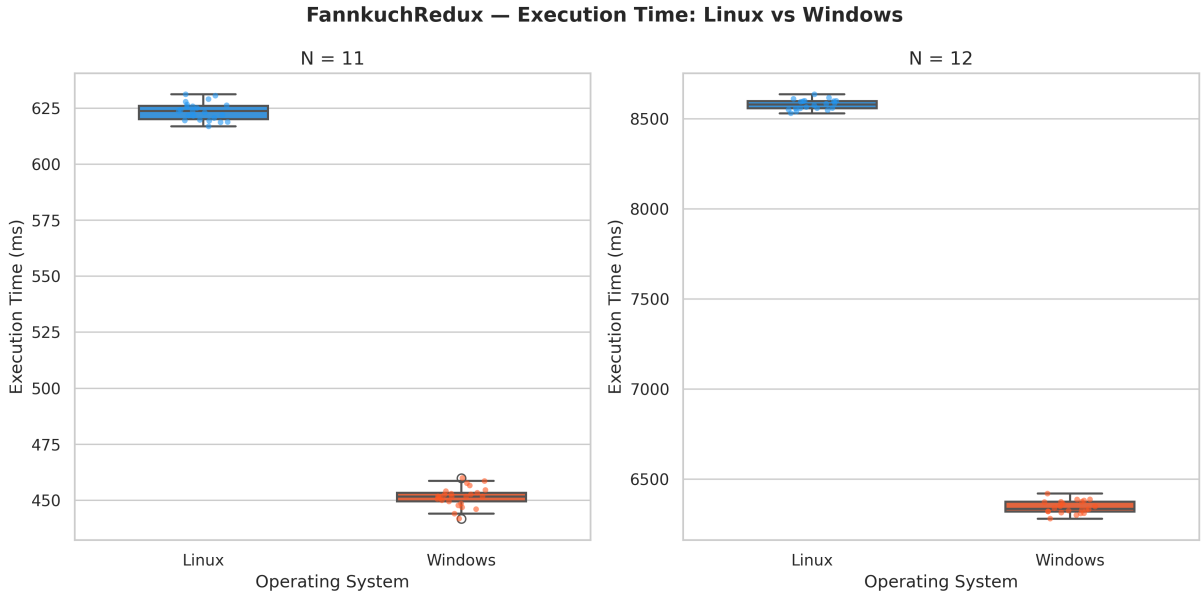

Figura 1: Distribuição do tempo de execução para N=11 e N=12 em Linux e Windows. Os pontos representam iterações individuais.

A Figura 2 apresenta a distribuição do consumo energético por SO e N.

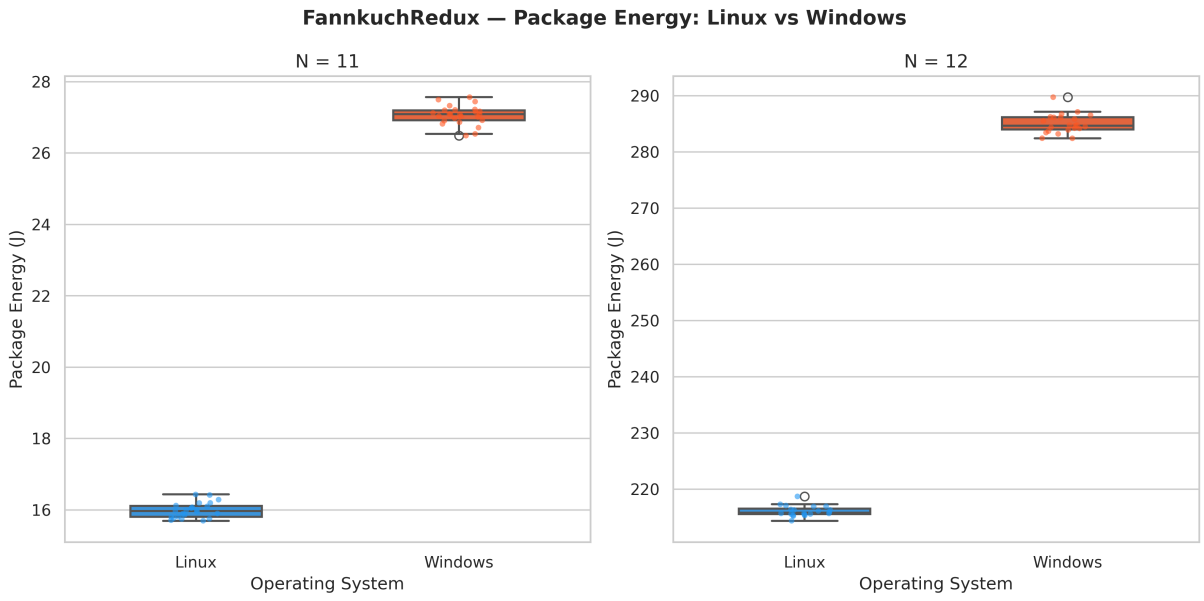

Figura 2: Distribuição do consumo energético (domínio Package) para N=11 e N=12.

A Figura 3 apresenta as médias e intervalos de confiança a 95% para tempo de execução e energia.

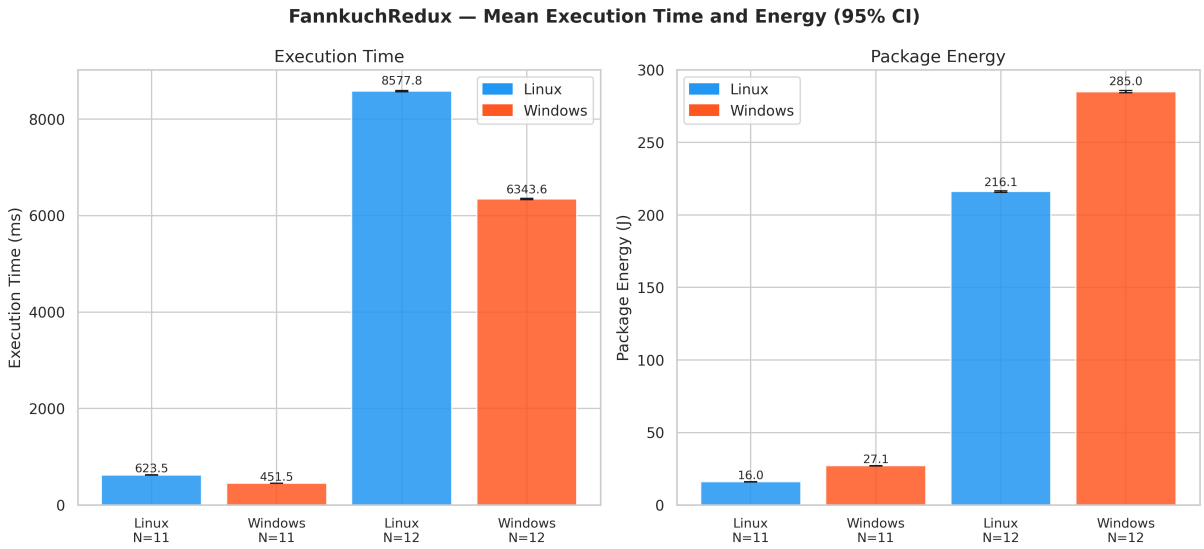

Figura 3: Média e IC 95% para tempo de execução (esq.) e energia (dir.) por SO e N.

A Figura 4 ilustra a correlação de Spearman entre tempo de execução e energia consumida.

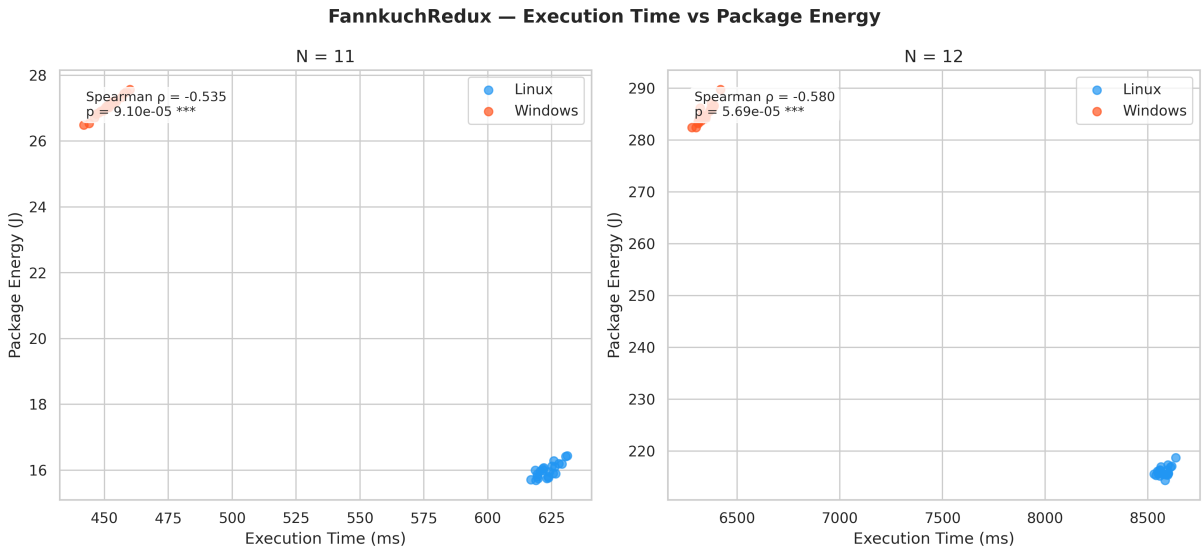

Figura 4: Correlação entre tempo de execução e energia consumida por SO (Spearman ρ).

4.3 Comparação de Performance (Tempo)

Para N=11, Windows executa em 451.5 ms (IC 95%: [449.7, 453.3] ms) vs. Linux 623.5 ms (IC 95%: [621.8, 625.2] ms). **Windows é 38.1% mais rápido.** Os intervalos de confiança não se sobrepõem.

Para N=12, Windows executa em 6343.6 ms (IC 95%: [6328.2, 6359.0] ms) vs. Linux 8577.8 ms (IC 95%: [8564.7, 8591.0] ms). **Windows é 35.2% mais rápido.**

4.4 Comparação de Consumo Energético

Para N=11, Linux consome 15.99 J (IC 95%: [15.90, 16.08] J) vs. Windows 27.06 J (IC 95%: [26.96, 27.17] J). **Linux consome 40.9% menos energia.**

Para N=12, Linux consome 216.14 J (IC 95%: [215.70, 216.58] J) vs. Windows 285.03 J (IC 95%: [284.28, 285.79] J). **Linux consome 24.2% menos energia.**

4.5 Significância Estatística

A Tabela 3 apresenta os resultados dos testes de hipótese. Todas as diferenças são estatisticamente significativas com $p < 0.001$ ($\alpha = 0.05$).

Tabela 3: Resultados dos testes de hipótese para tempo e energia.

N	Métrica	Teste	Estatística	p-value	Significativo
11	Tempo	Mann-Whitney U	575.0	<0.001	Sim (***)
11	Energia	Mann-Whitney U	0.0	<0.001	Sim (***)
11	Tempo	Permutação A/B	172.0 ms	<0.001	Sim (***)
11	Energia	Permutação A/B	−11.07 J	<0.001	Sim (***)
12	Tempo	Mann-Whitney U	440.0	<0.001	Sim (***)
12	Energia	Mann-Whitney U	0.0	<0.001	Sim (***)
12	Tempo	Permutação A/B	2234.2 ms	<0.001	Sim (***)
12	Energia	Permutação A/B	−68.89 J	<0.001	Sim (***)

4.6 Correlação Tempo–Energia

A correlação de Spearman entre tempo de execução e energia consumida é forte e estatisticamente significativa em ambos os grupos:

- Spearman ρ (dataset completo) $= 0.607$, $p < 0.001$
- Spearman ρ (Linux) $= 0.895$, $p < 0.001$
- Spearman ρ (Windows) $= 0.983$, $p < 0.001$

Este resultado é esperado para um *workload* CPU-bound puro. A correlação mais forte no Windows sugere comportamento mais previsível na relação energia/tempo nesse SO.

4.7 Variância e Estabilidade

O coeficiente de variação (CV) do Linux é consistentemente abaixo de 1% (0.33%–0.63%), indicando ambiente de medição muito estável. O CV do Windows N=11 (0.95%) é superior ao Linux, refletindo maior variabilidade do *scheduler* do Windows para *workloads* de curta duração. O CV do Windows N=12 (0.55%) normaliza para valores próximos do Linux, sugerindo que *workloads* mais longas estabilizam o comportamento.

A Figura 5 mostra a distribuição do tempo de execução por SO e N.

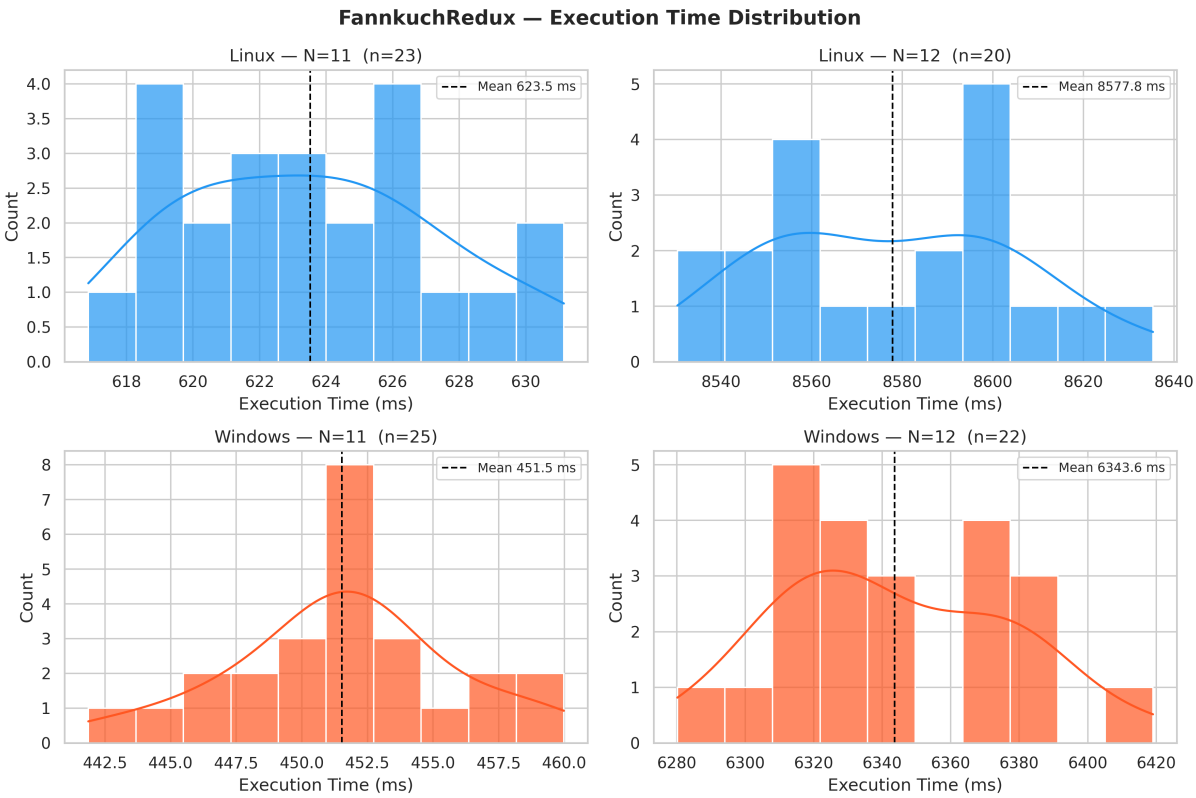

Figura 5: Distribuição do tempo de execução por SO e N (histograma + KDE).

5 Discussão

5.1 Windows é mais rápido — porquê?

O algoritmo usa `Task.Run` com `Environment.ProcessorCount+1` *workers*. O ThreadPool do Windows é historicamente mais agressivo a escalonar *threads* em múltiplos núcleos para *workloads* curtas. O .NET no Windows pode também beneficiar de otimizações específicas da plataforma no JIT (RyuJIT x64). Embora o turbo tenha sido desativado, diferenças residuais de frequência entre os governadores de cada SO não podem ser completamente excluídas. O *scheduler* CFS do Linux (*Completely Fair Scheduler*) prioriza equidade sobre *throughput* máximo, o que pode desfavorecer *workloads* multi-threaded intensivas.

5.2 Linux consome menos energia — porquê?

Windows executa mais rápido, mas consome proporcionalmente muito mais energia. Para N=11, Windows é 38% mais rápido mas consome 69% mais energia. Este resultado é contra-intuitivo se se assumir que "mais rápido = menos tempo a trabalhar = menos energia". A explicação mais provável é que Windows opera os núcleos a voltagem/frequência mais elevada mesmo com turbo desativado (estados de potência C0 mais agressivos, menor uso de C-states entre tarefas). O governor `performance` do Linux mantém frequência base constante (≈2.6 GHz) com transições suaves de C-state. O plano "Desempenho Máximo" do Windows pode operar em estados de potência mais elevados de forma mais contínua. Para *workloads* onde o tempo de resposta não é crítico, Linux oferece melhor eficiência energética.

5.3 Trade-off velocidade vs. energia

Existe um trade-off claro: Windows oferece aproximadamente 35–38% mais velocidade em troca de aproximadamente 25–69% mais consumo energético. A magnitude do trade-off varia com N: para N=12 (*workload* mais longa), a diferença de energia reduz-se (24% vs. 69% para N=11), sugerindo que o efeito é mais pronunciado em *workloads* curtas onde os *overheads* de *scheduling* dominam. Para aplicações onde a eficiência energética é prioritária (e.g., servidores em *data centers* com custo de energia elevado, dispositivos embebidos), Linux apresenta vantagem clara.

6 Ameaças à Validade

6.1 Validade Interna

Sessões separadas: as medições Linux e Windows foram realizadas em momentos distintos. Fatores como temperatura ambiente e estado do hardware podem introduzir *bias* sistemático.

Temperatura Windows não disponível: impossibilidade de correlacionar temperatura com desempenho no Windows.

Filtro RAPL: a Intel adicionou ruído aleatório às leituras RAPL (IPU 2021.2) como mitigação de ataques *side-channel*. Afeta ambos os SO de forma equivalente; mitigado pelo número elevado de iterações.

6.2 Validade Externa

Hardware único: os resultados foram obtidos num único *laptop* (HP EliteBook 1050 G1, i7-8850H) e podem não generalizar para outros processadores (AMD, ARM) ou fatores de forma (desktop, servidor).

Algoritmo único: FannkuchRedux é um *benchmark* CPU-bound puro. Resultados podem diferir para *workloads* com I/O intensivo, uso intensivo de memória ou execução *single-threaded*.

Versão .NET: testado com .NET 9. Resultados podem variar com versões anteriores ou futuras.

6.3 Validade de Construção

Dimensão da amostra: 20–25 amostras por grupo estão ligeiramente abaixo do $n = 30$ recomendado pelo Teorema do Limite Central. Os testes não-paramétricos utilizados (Mann-Whitney, permutação) são válidos para $n \geq 20$.

Variância Windows N=11: CV $\approx$ 0.95% indica comportamento menos determinístico. Os *outliers* foram removidos pelo critério IQR antes dos testes.

7 Conclusão

Este trabalho comparou a performance e o consumo energético de C# (.NET 9) em Windows 11 e Ubuntu 24.04 usando o algoritmo FannkuchRedux como *benchmark* CPU-bound. Os resultados mostram que Windows 11 é consistentemente mais rápido (35–38%) mas consome significativamente mais energia (25–69%) que Ubuntu 24.04, nas mesmas

condições de hardware. Todas as diferenças são estatisticamente significativas (Mann-Whitney U, $p < 0.001$) e confirmadas por testes de permutação independentes.

Os resultados evidenciam um trade-off claro entre velocidade e eficiência energética dependente do SO. Para aplicações onde o custo energético é prioritário — como servidores em larga escala ou sistemas embebidos — Linux apresenta vantagem significativa. Como trabalho futuro sugere-se: (1) replicar o estudo noutros tipos de hardware (desktop, servidor ARM); (2) incluir *workloads single-threaded* e com I/O intensivo; (3) comparar versões de .NET (8 vs. 9 vs. 10); (4) investigar o impacto de diferentes configurações do *scheduler* Linux.

Referências

[1] BenchmarkDotNet contributors. BenchmarkDotNet. GitHub, 2024. URL https://github.com/dotnet/BenchmarkDotNet.

[2] greensoftwarelab. Energy-languages. GitHub, 2017. URL https://github.com/greensoftwarelab/Energy-Languages.

[3] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3B: System Programming Guide, Part 2*, 2023. Chapter 14: RAPL — Running Average Power Limit.

[4] LibreHardwareMonitor contributors. LibreHardwareMonitor. GitHub, 2024. URL https://github.com/LibreHardwareMonitor/LibreHardwareMonitor.

[5] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, SLE 2017, pages 256–267. ACM, 2017. doi: 10.1145/3136014.3136031.

[6] The Computer Language Benchmarks Game contributors. Computer language benchmarks game. https://benchmarksgame-team.pages.debian.net/benchmarksgame/, 2024.